

COSC2129/1476

Artificial Intelligence

Solving Problems by Searching

Lecturer: Thuy Nguyen

Email: thuy.nguyen43@rmit.edu.vn

Office location: SGS 2.4.27

Road Map for Today

Part 1 : Introduction to Search & Uninformed Search

Part 2 : Informed Search

Part I

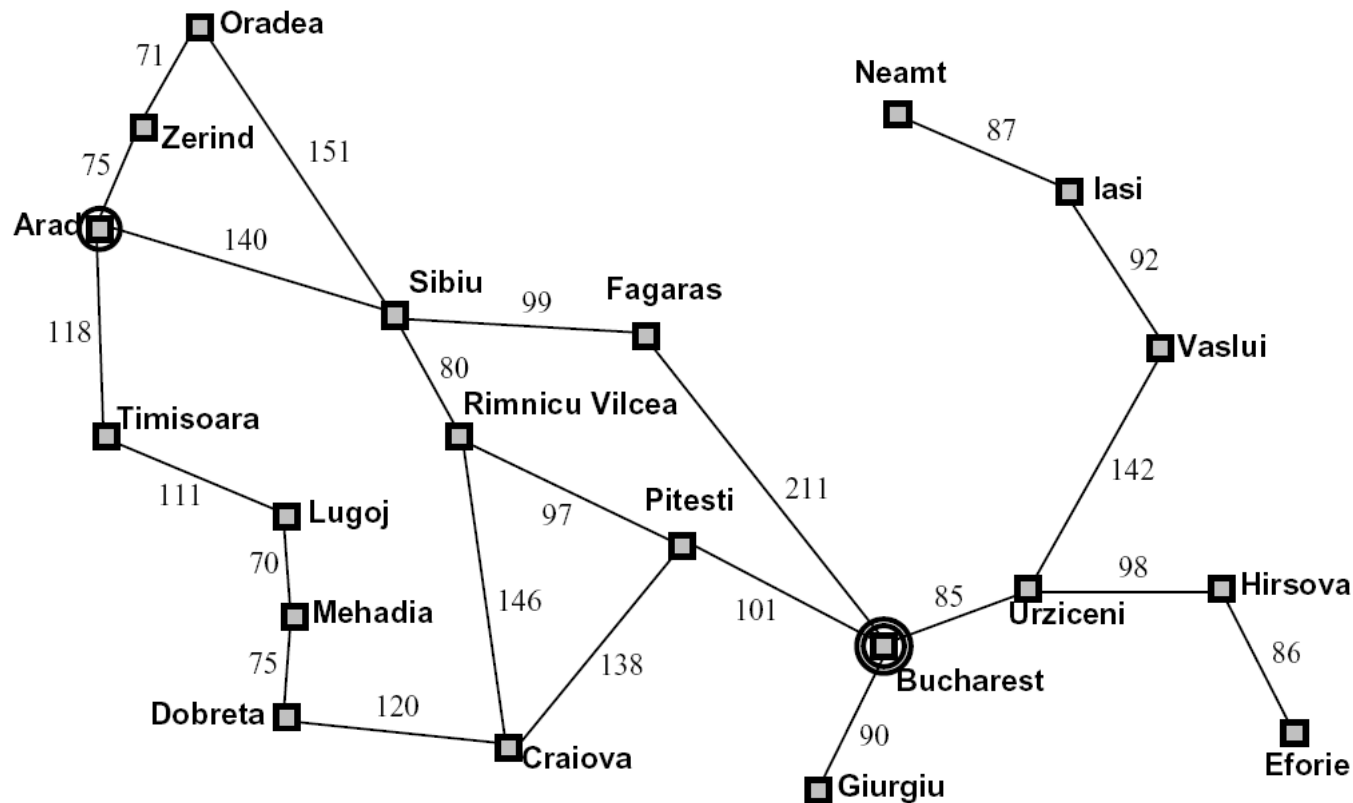
Introduction to Search & Uninformed Search

What is Search?

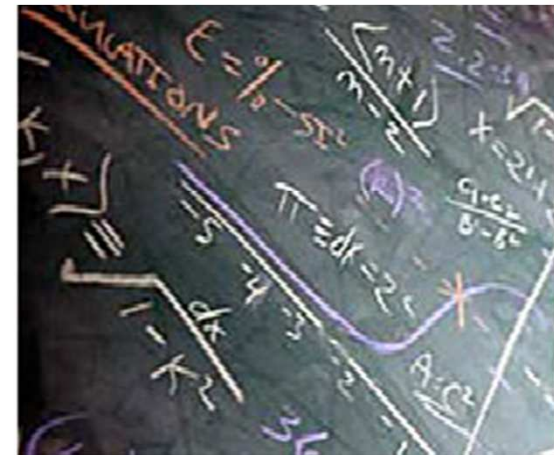
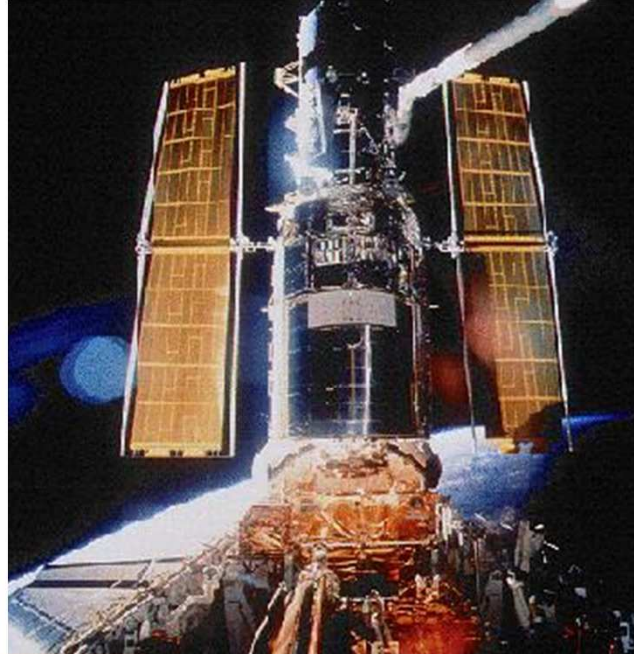
- ❑ Search is an exploration of possibilities.
- ❑ Particularly useful when a “direct” method/algorithm is not known.
- ❑ One of the first topics studied in AI:
 - Newell and Simon (1961): General Problem Solver.
- ❑ Central component to many AI systems:
 - Automated reasoning, theorem proving, robot navigation, VLSI layout, scheduling, game playing, etc.

Example: A Route-Finding Problem

- ❑ In deciding to travel in Romania, say from **Arad** to **Bucharest**, we would look at different routes on a map before choosing one.
- ❑ How many possible routes are there?



More Examples

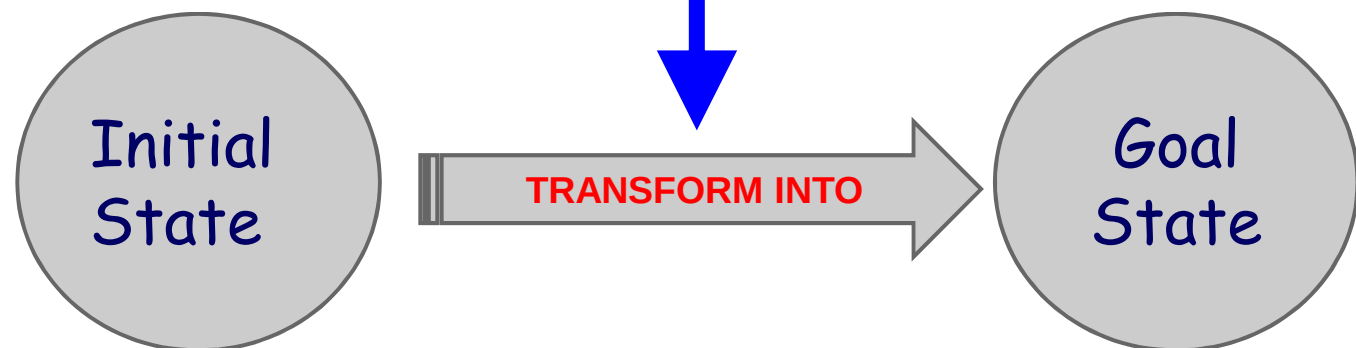


Search Problems

A search problem is defined by:

- ❑ Possible states (search space)
- ❑ Initial state
- ❑ Actions
- ❑ Transition model
- ❑ Goal test
- ❑ Path cost

How to do this?

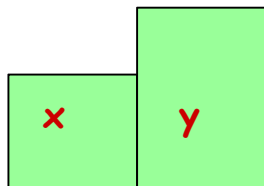


Water Jugs

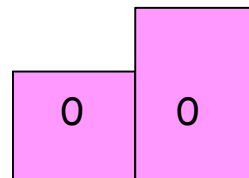
Given two water jugs
(of different sizes),
get exactly x litres of
water in one of the
jugs



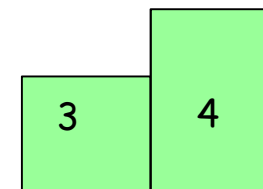
Generic "state":



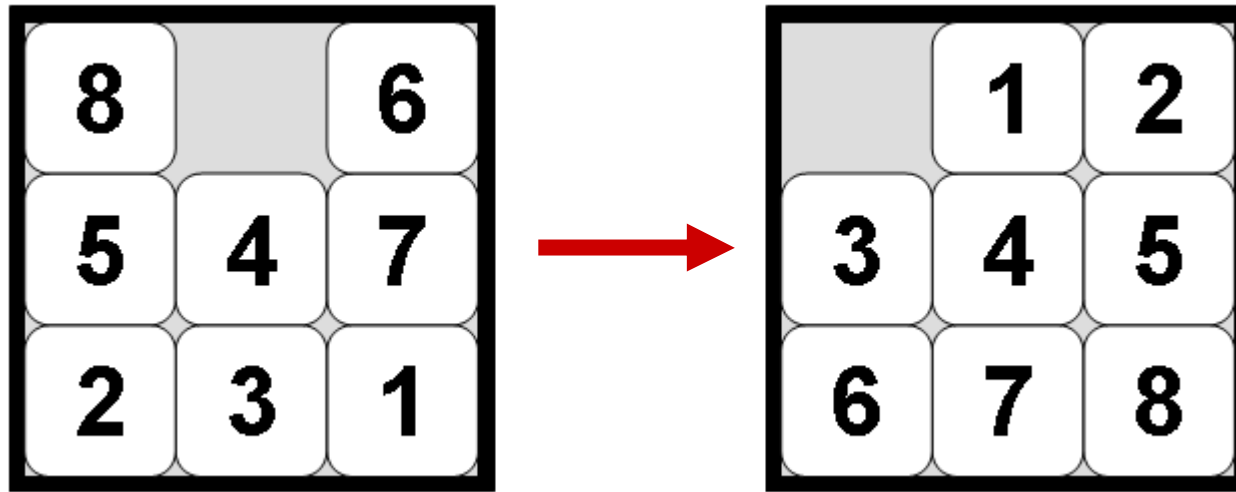
Initial "state":



Final "state":

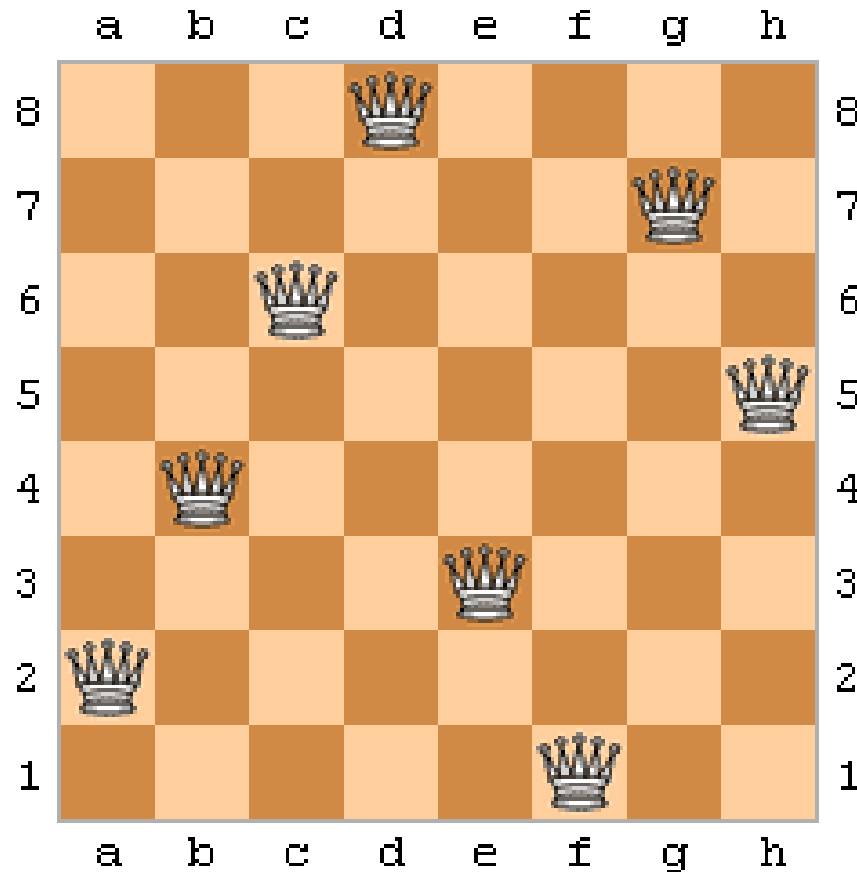


8-Puzzle



One of sliding-block puzzles, often used as test problems for new search algorithms in AI

N-Queens



Place N (usually 8) queens on a chessboard so that no queen attracts any other.

Vacuum World

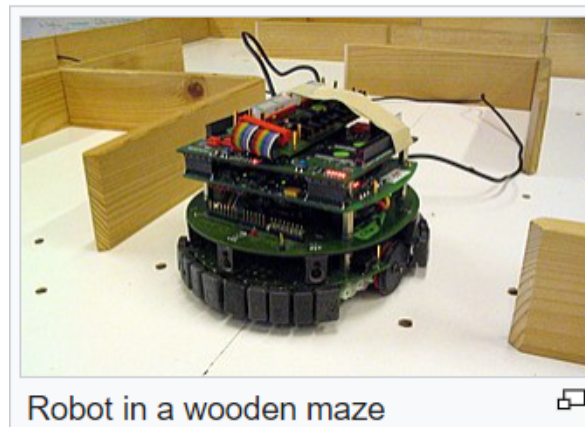
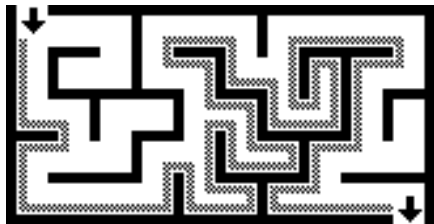


- ❑ Two rooms
 - One vacuum
 - Rooms can be **dirty** or **clean**
 - Vacuum has three moves:
 - ✓ Left
 - ✓ Right
 - ✓ Suck
- ❑ Only 8 possible states

Problem solving as search

Goal-based agents

- Agents that work towards a **goal**.
- Agents consider the impact of **actions** on future **states**.
- Agent's job is to identify the action or series of actions that lead to the goal.
- Formalized as a **search** through possible **solutions**.

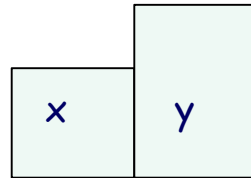


Problem formulation

- **Initial state:** the state in which the agent starts
- **States:** All states reachable from the initial state by any sequence of actions (State space)
- **Actions:** possible actions available to the agent. At a state s , $\text{Actions}(s)$ returns the set of actions that can be executed in state s . (Action space)
- **Transition model:** A description of what each action does
 $\text{Results}(s, a)$
- **Goal test:** determines if a given state is a goal state
- **Path cost:** function that assigns a numeric cost to a path w.r.t. performance measure.

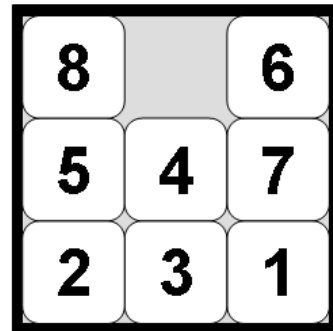
State

Water Jugs:

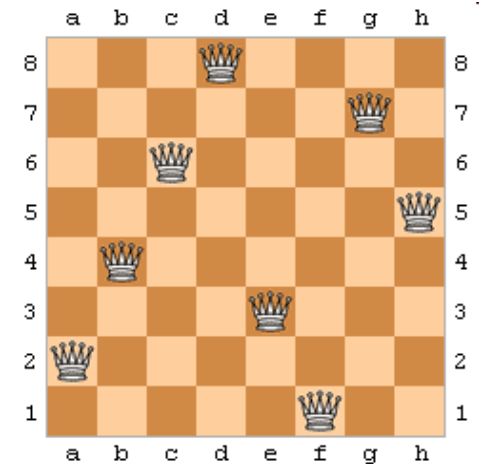


$x \in \{0,1,2,3\}$
 $y \in \{0,1,2,3,4,5\}$

Eight Puzzle:



N-Queens:



Vacuum
World:



Initial State

- . Water Jugs: both jugs are empty
- . Eight Puzzle: any state
- . N-Queens: empty chessboard
- . Vacuum World: any state

Actions

Water Jugs:

- empty or fill either jug (from tap)
- pour jug 1 into jug 2
- pour jug 2 into jug 1

Eight Puzzle: move a tile towards left, right, up, down (not always possible)

N-Queens:

- add queen to any empty square
- add queen to leftmost empty column
- add queen to leftmost empty column in a "safe" row

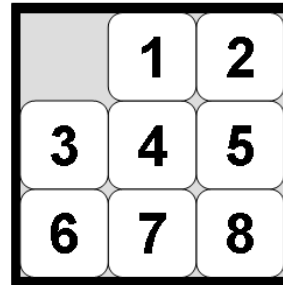
Vacuum World: left, right, suck

Transition Model

- . Water Jugs: new state of jugs
- . Eight Puzzle: new puzzle state (if action applicable)
- . N-Queens: new chessboard
- . Vacuum World: new room state

Goal Test

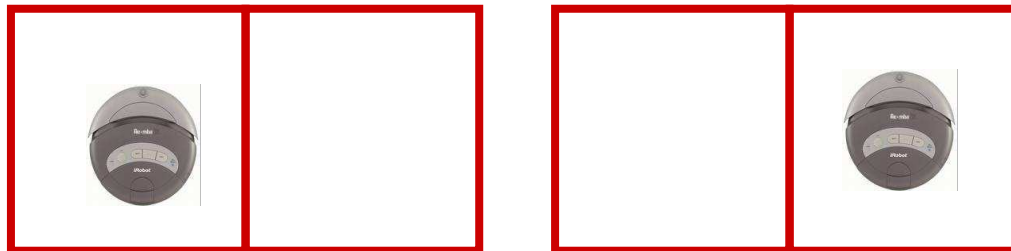
Water Jugs: 4 litres in the larger jug



Eight Puzzle:

N-Queens: All N queens on board in safe positions

Vacuum World:



Path Cost

Water Jugs: number of exchanges

Eight Puzzle: number of tiles moves

N-Queens: number of piece moves

Vacuum World: number of moves

Search States

A **state** captures all relevant information about the problem at that moment, such as:

- ❑ The current location:

(At <city>)

[Route-Finding Problem]

Abstraction!

- ❑ The location of each queen:

(Arrangement of 0 to 8 queens on the board)

[8-Queens Problem]

Solving Problems by Searching

- ❑ We can formulate a search problem.
 - Now we need to find the solution.
- ❑ We can visualize a state space search in terms of trees or graphs.
 - Nodes correspond to states.
 - Edges correspond to taking actions (operations).
- ❑ We will be studying **search trees**.
 - These trees are constructed “on the fly” by search algorithms.

Example: Cannibals & Missionaries

Three cannibals and three missionaries come to a crocodile infested river. There is a boat on their side that can be used by one or two persons.

If cannibals outnumber the missionaries at any time, *the cannibals will eat the missionaries*. How can they use the boat so that everyone survives?

State representation:

(#C-on-left, #M-on-left,
boat-at-bank, #C-on-right, #M-on-right)

Initial state: (3, 3, LEFT, 0, 0)

Goal state: (0, 0, RIGHT, 3, 3)

Example: Cannibals & Missionaries (cont'd)

An **action (operation)** could be written as:

`Move(#M, #C, source-bank, dest-bank)`

For example:

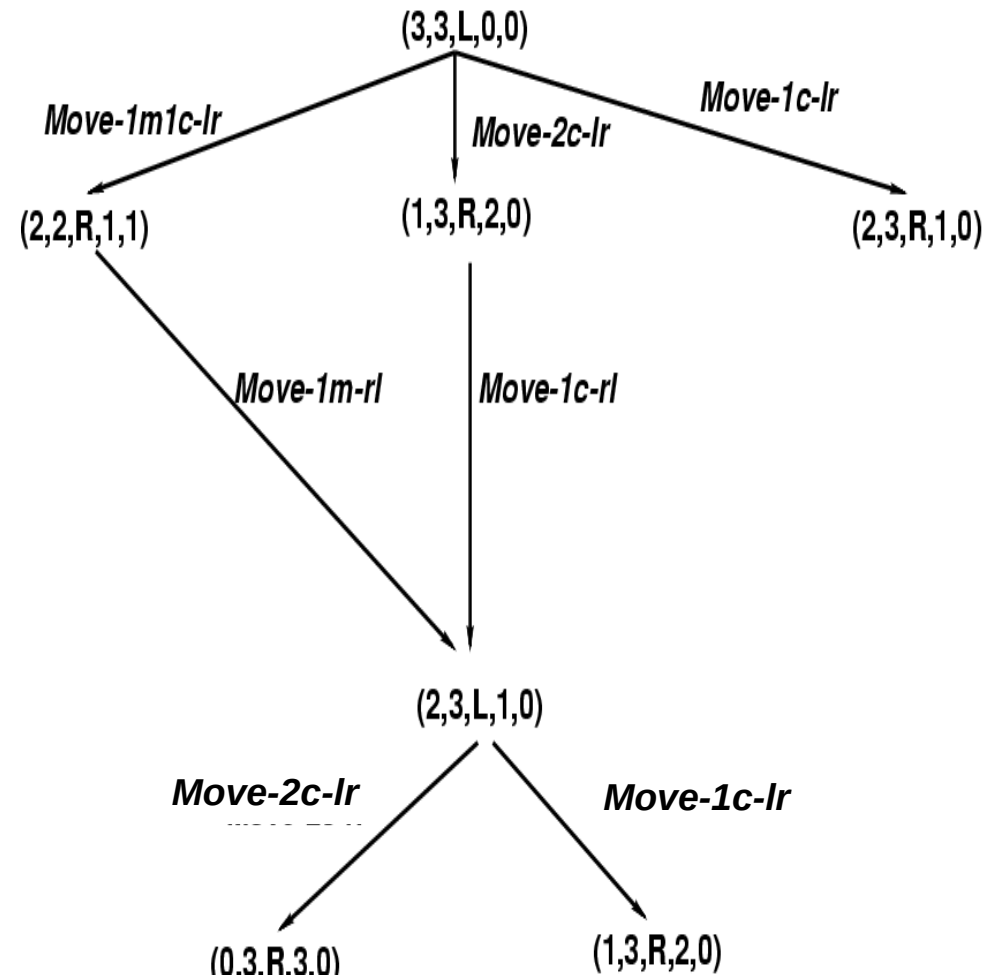
`Move(1, 1, left, right)`

or

`Move-1m1c-lr`

Meaning: Moving 1 missionary and 1 cannibal from the left bank to the right bank

Fragment of the State Space



Searching the State Space

- ❑ State space searching involves **finding a path** from the initial state to the goal state.
 - Building a **search tree** starting from the initial state
 - **Expanding a state** by applying actions (operations) to generate successor (child) state(s).
- ❑ The order in which we choose states for expansion is determined by the **search strategy**



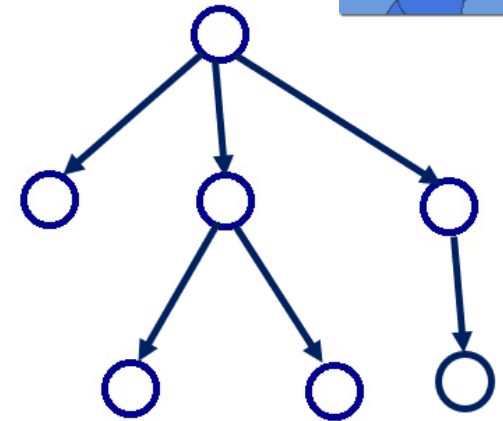
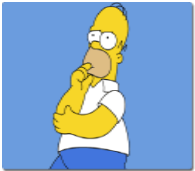
Generic Search Procedure

```
function SEARCH (problem, strategy)
    return solution or failure
```

Initialise the search tree with the initial state of the **problem**

Repeat

1. If (no candidate nodes can be expanded) then return failure
2. Choose a leaf node for expansion according to **strategy**
3. If (chosen node contains a goal state) then return solution
4. Else expand the node, by applying legal actions (operations) to the state of the node, and then adding newly generated node(s) to the search tree.



Implementation Details

Need to keep track of nodes that need to be expanded by using a data structure called "QUEUE"

```
function SEARCH (problem, strategy)
  return solution or failure
```

Search algorithms differ in their queuing function!

Initialize a **queue** by inserting the node corresponding to the initial state of the problem

Repeat

1. If (the **queue** is empty) then return failure
2. **Dequeue** a node (//dependent on the search strategy)
3. If (this node contains a goal state) then return solution
4. Else

Expand the node, by applying legal actions (operations) to the state of the node, and add newly generated node(s) to the **queue**.

Data Structures for Search

Node

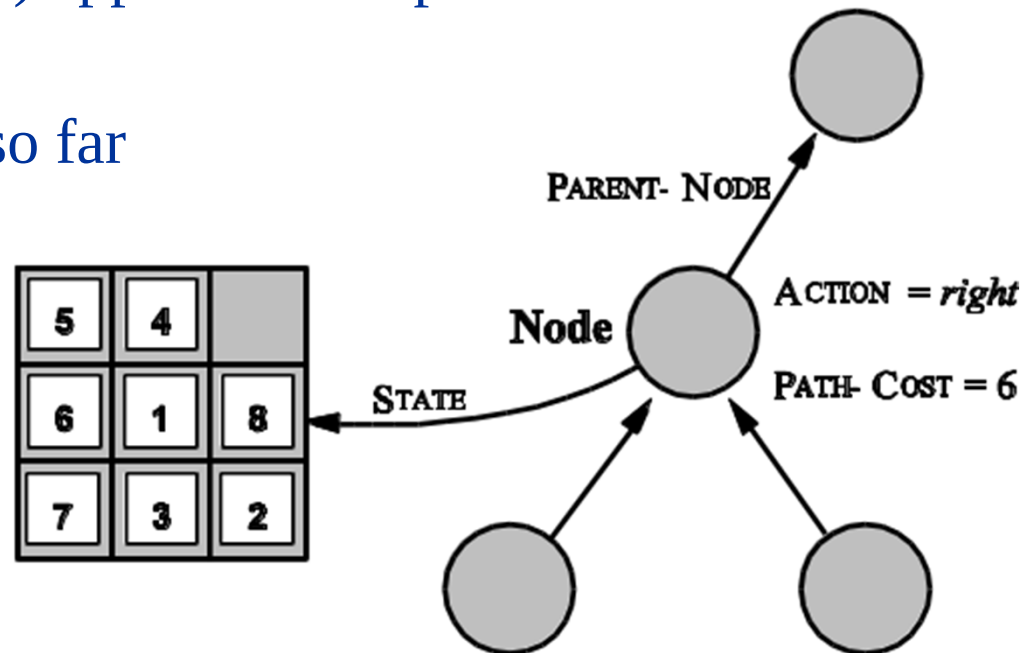
- ❑ State
- ❑ Parent node
- ❑ Action (operation) applied to the parent node to reach the current node
- ❑ Cost of the path so far

Queue

- ❑ Frontier

Hash Table

- ❑ Efficient checking for repeated states



Search Performance

Completeness: Is it guaranteed to find a solution if one exists?

Optimality: Will it find an optimal solution?

Time complexity: How many nodes get expanded?

Space complexity: How much memory is needed?

b	Branching factor
d	Depth of shallowest goal node
m	Maximum length of any path in the state space

Search Types

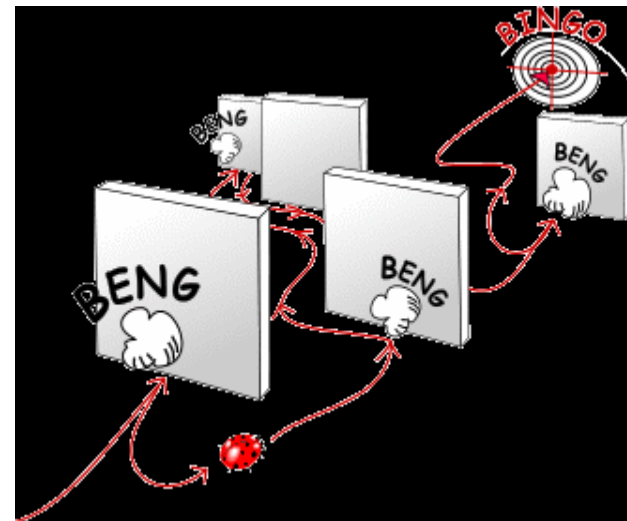
Uninformed (blind) search

- ❑ Breadth-first
- ❑ Depth-first
- ❑ Uniform cost
- ❑ Depth-limited
- ❑ Iterative Deepening



Informed (heuristic) search

- ❑ Greedy Best-first search
- ❑ A^*

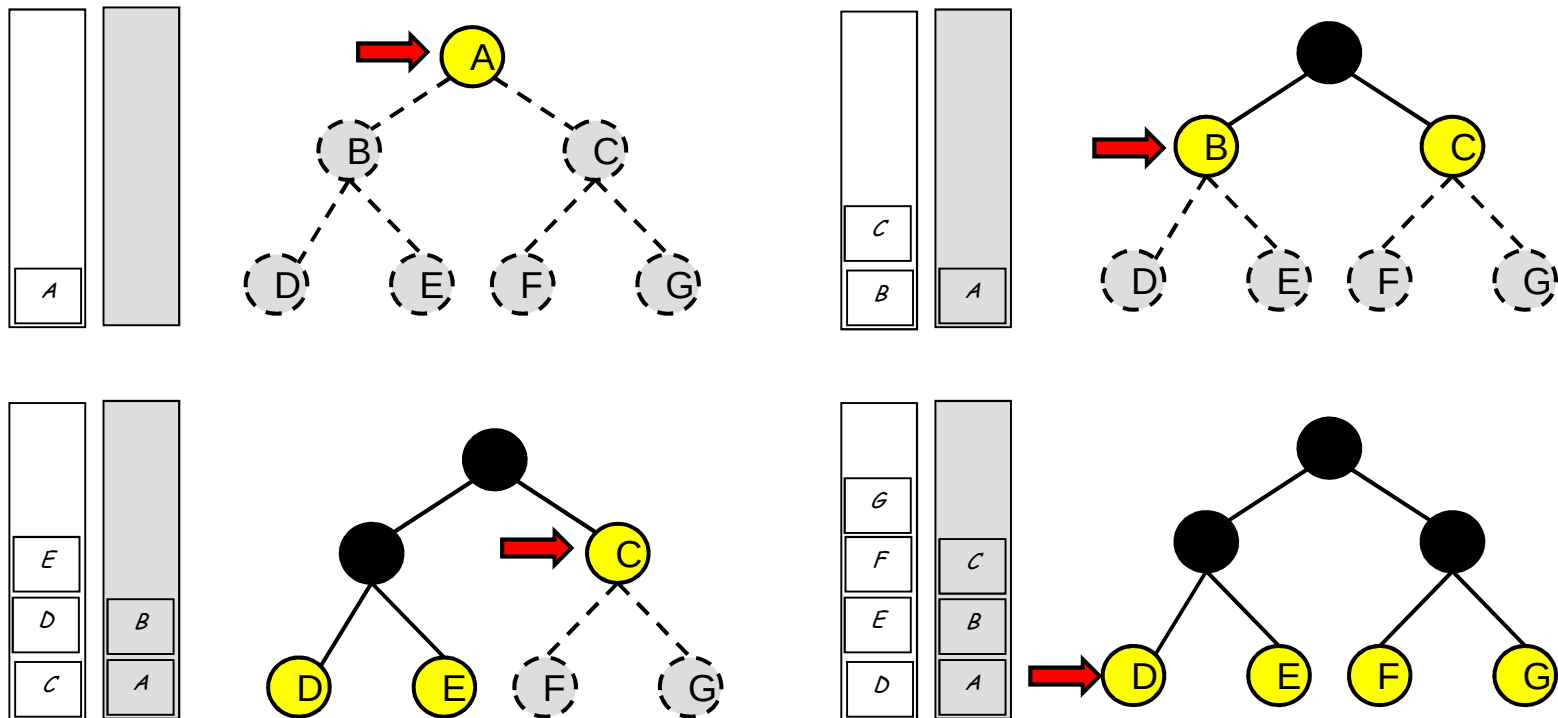


Breath-First Search (BFS)

- ❑ Frontier is a **FIFO queue**.
- ❑ Shallowest unexpanded node first (**shallowest depth of tree $\neq$ lowest path cost**)
- ❑ Discards any new path to a state in the frontier or explored set (new path must be longer, so ...)
- ❑ Has shallowest path to any node in frontier

An Example of BFS

Nodes are explored in order: A,B,C,D,E,F,G



Judging BFS

- ❑ Complete:

Yes, if b is finite

- ❑ Optimal:

Yes, if all step costs are the same

- ❑ Time:

$$1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$$

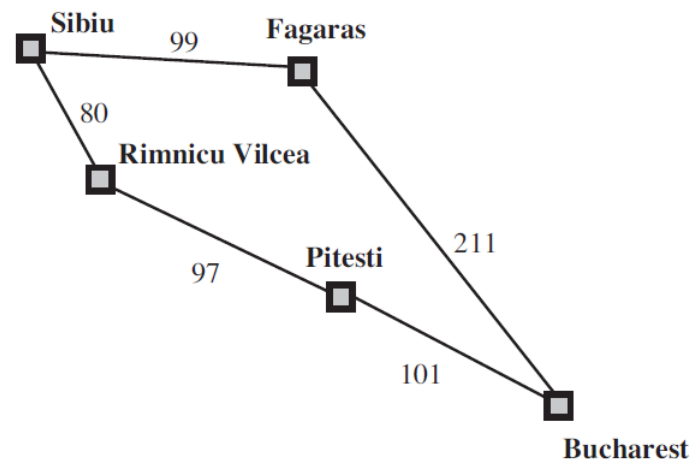
- ❑ Space:

$$O(b^d)$$

All uninformed search methods will have exponential time complexity.. :-)

Uniform-Cost Search

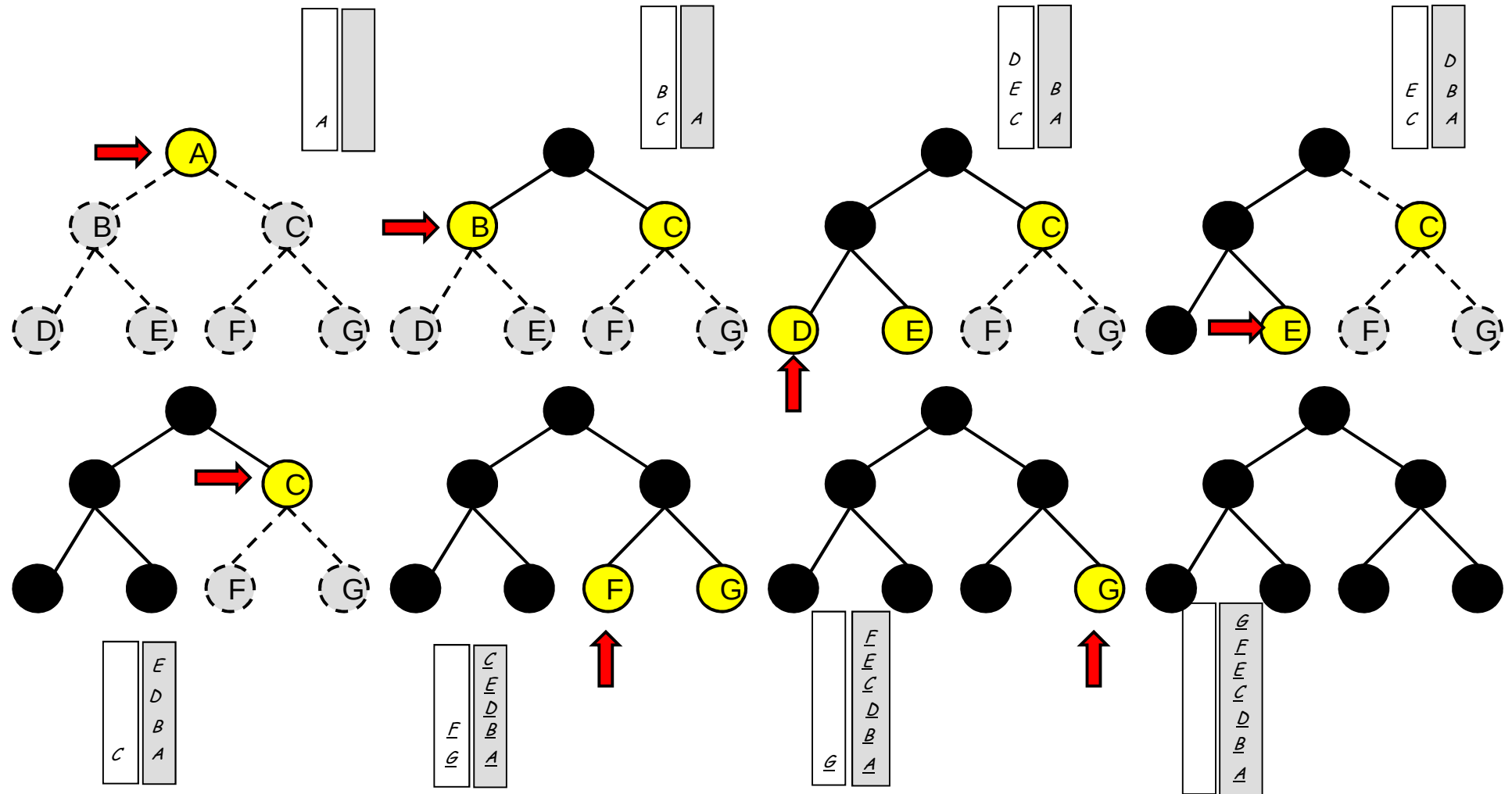
- Frontier is a **priority queue** ordered by **path cost $g(n)$**
e.g., $g(n)$ in a route-finding problem is the sum of distances between cities in the path to n from the root.
- Test goal when selected for expansion
- Variation of BFS. Same if **$g(n) = \text{depth}(n)$**



Depth-First Search (DFS)

- ❑ DFS always expands the deepest node in the current frontier.
- ❑ Frontier is constructed as a **stack** (LIFO queue) .
- ❑ Deepest unexpanded node first
- ❑ Moves to next deepest when node with no successor is found

An Example of DFS



Nodes explored in order: A,B,D,E,C,F,G

Judging DFS

- ❑ Complete:

No in general (unless cycles are checked)

- ❑ Optimal:

No, might return a solution which is deeper (i.e. more costly) than another solution

- ❑ Time:

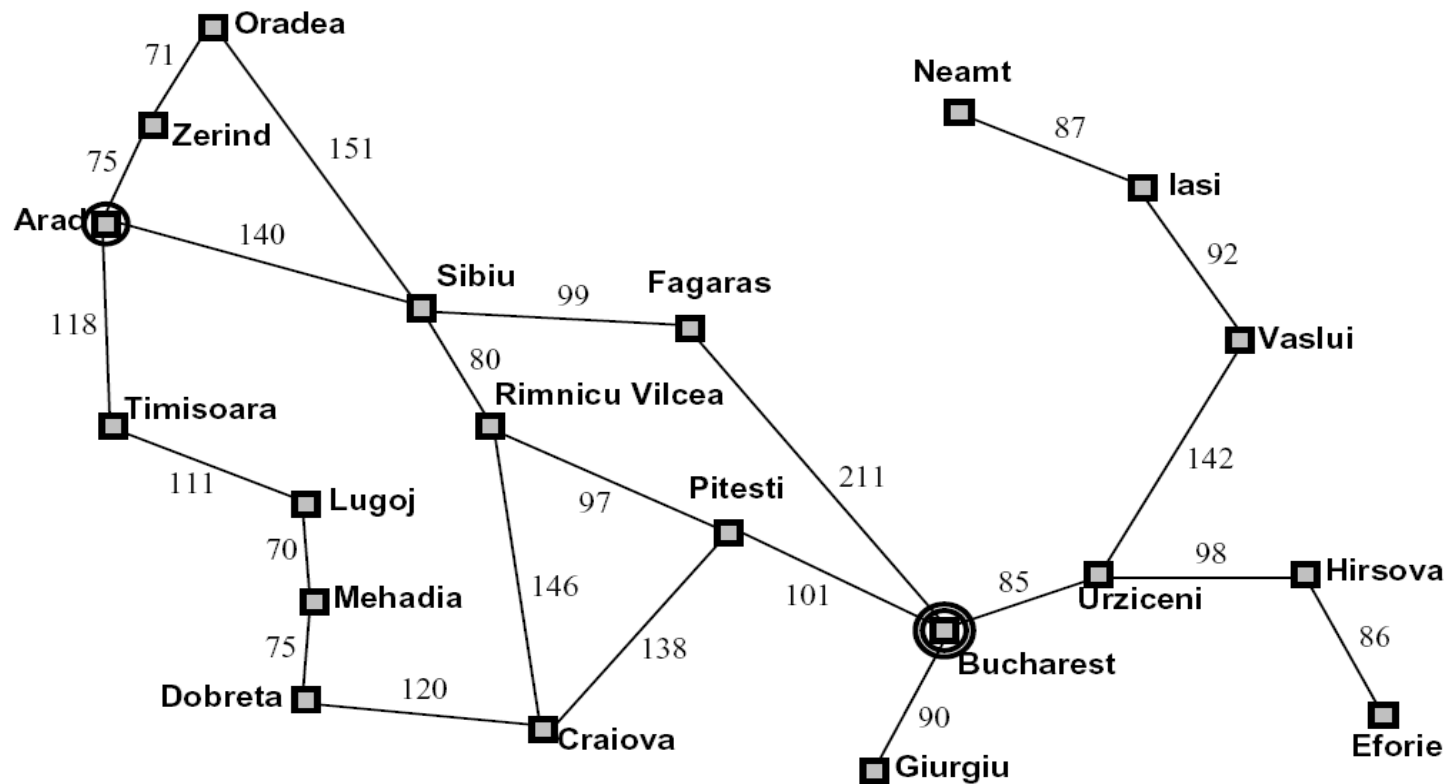
$O(b^m)$, m might be larger than d .

- ❑ Space:

$O(bm)$

An Example for BFS and DFS

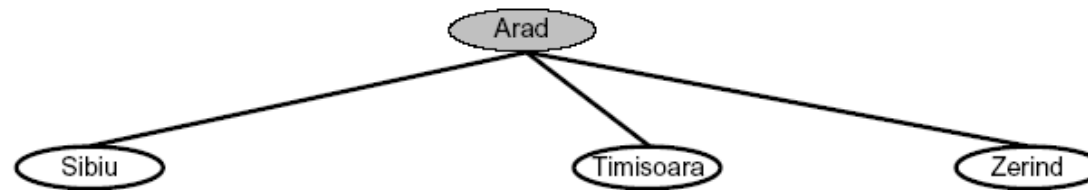
- Travelling from Arad to Bucharest:



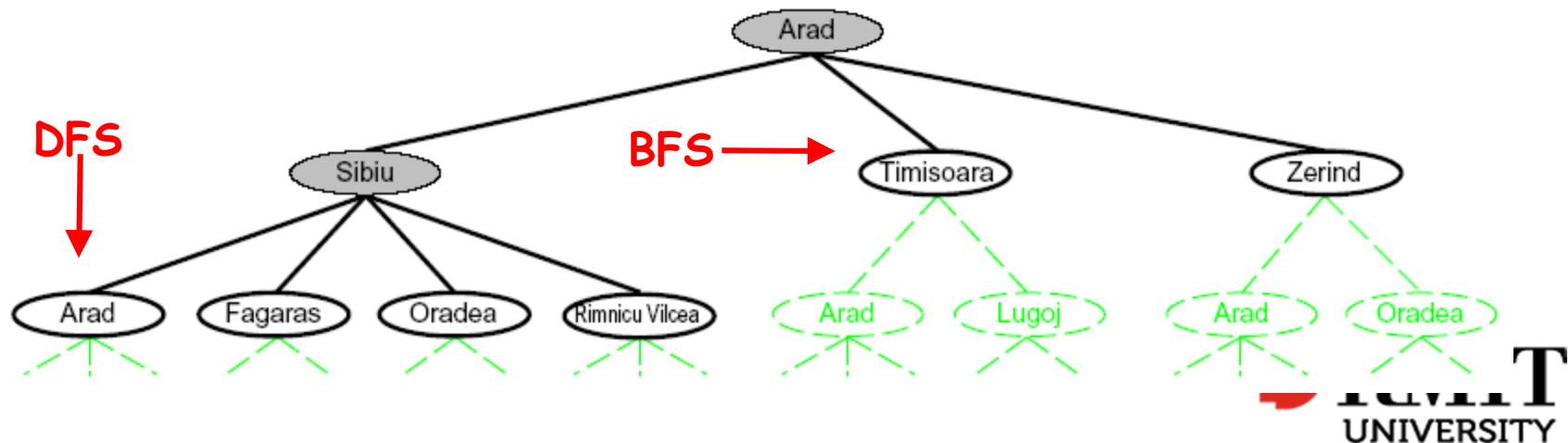
- A state could be represented as being in a city.

An Example for BFS and DFS (cont'd)

- After expanding Arad:



- After expanding Sibiu:
 - BFS will expand Timisoara next.
 - DFS will expand Arad next.



Depth-Limited Search (DLS)

- ❑ DLS is depth-first search (DFS) with depth bounded.
- ❑ We can avoid the problem of unbounded trees by using a **depth limit l** .
 - All nodes at depth l are treated as if they have no successors.
 - If possible, l should be based on knowledge of the problem
- ❑ Time: $O(b^l)$
- ❑ Memory: $O(bl)$
- ❑ Complete?: No
- ❑ Optimal?: No

DLS terminates with two kinds of failure:

- ❑ No solution (standard failure)
- ❑ No solution within the depth limit (cutoff)

Iterative Deepening Search (IDS)

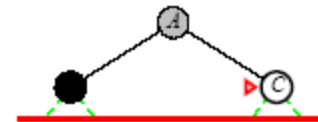
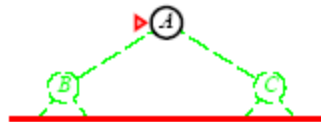
- ❑ IDS is a general strategy that repeatedly does depth-limited search, but **increases the depth limit each time**.
- ❑ IDS combines the advantages of depth-first and breadth-first search.
- ❑ IDS is, in general, the preferred uninformed search method when the search space is large and the solution depth is unknown.

IDS (cont'd)

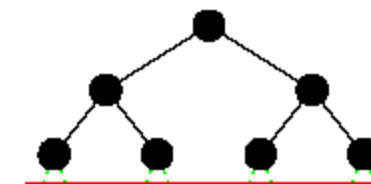
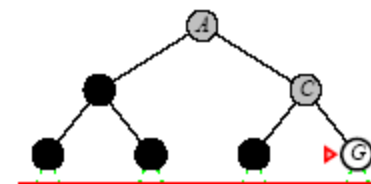
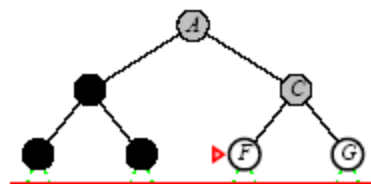
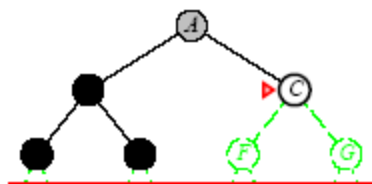
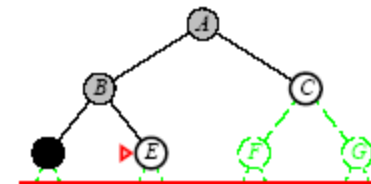
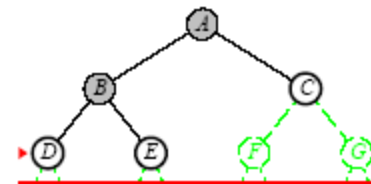
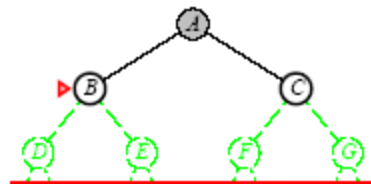
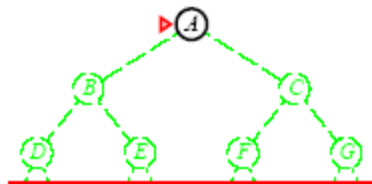
- DLS with depth limit: 0



- DLS with depth limit = 1

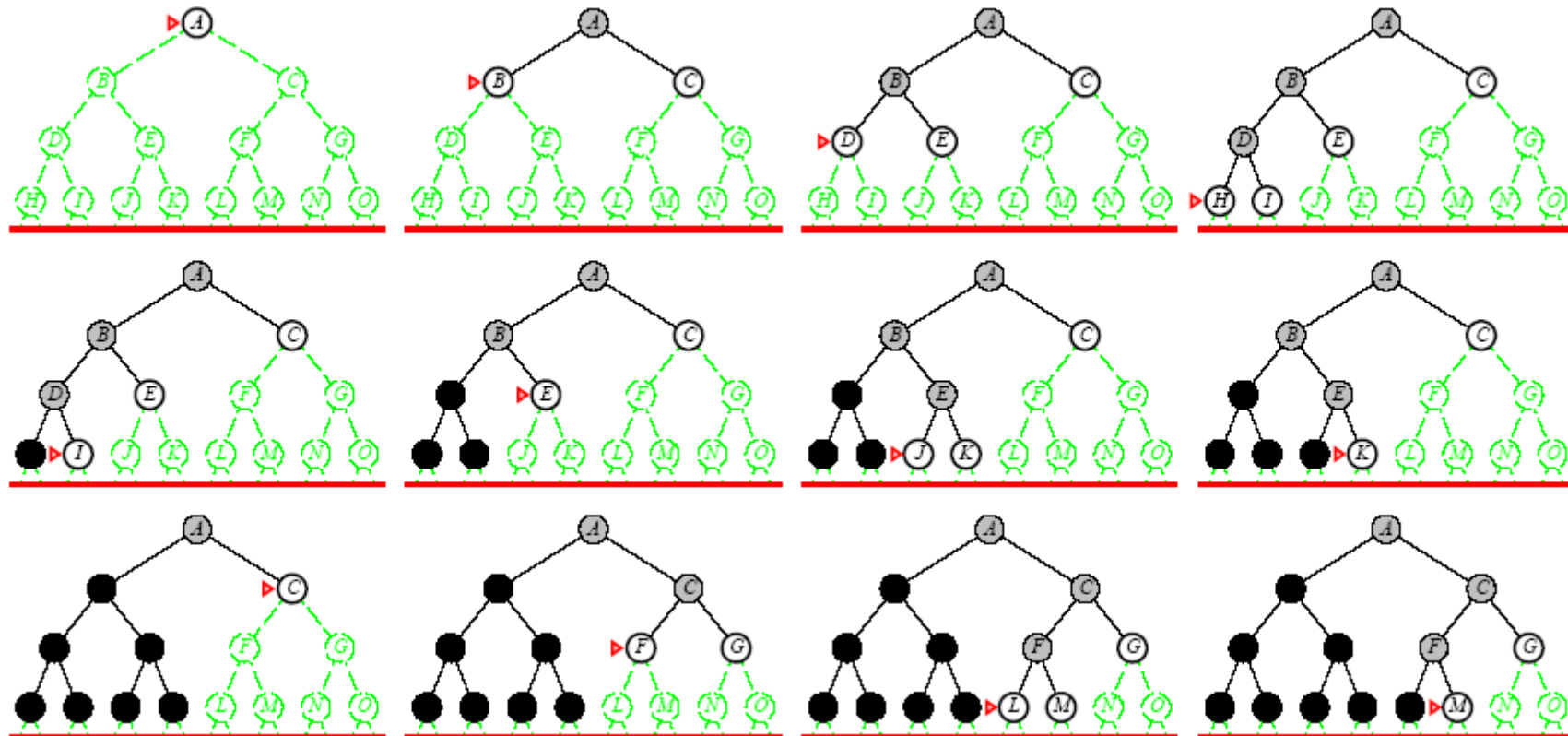


- DLS with depth limit = 2



IDS (cont'd)

- DLS with depth limit = 3



Judging IDS

- ❑ IDS is not as wasteful as one might think.
- ❑ Note, most nodes in a tree are at the bottom level.

Breadth-first search:

Time complexity: $1 + b + b^2 + \dots + b^{(d-1)} + b^d$

e.g. $b=10, d=5$: $1+10+100+1,000+10,000+100,000 = 111,111$

Iterative deepening search:

Time complexity: $(d+1)*1 + (d)*b + (d-1)*b^2 + \dots + 2b^{(d-1)} + 1b^d$

e.g. $b=10, d=5$: $6+50+400+3000+20,000+100,000 = 123,456$

Complete, Optimal, $O(b^d)$ time, $O(bd)$ space

Comparing BFS, DFS and IDS

- ❑ Breadth-first search (BFS) is complete and optimal, but has high space complexity.
- ❑ Depth-first search (DFS) is space-efficient, but is neither complete nor optimal.
- ❑ Iterative deepening search (IDS) is complete and optimal, with the same space complexity as DFS and almost the same time complexity as BFS.

Performance Comparision

Uninformed (blind) search

- ❑ Breath-first
- ❑ Depth-first
- ❑ Uniform-cost
- ❑ Depth-limited
- ❑ Iterative deepening

What about repeated states?

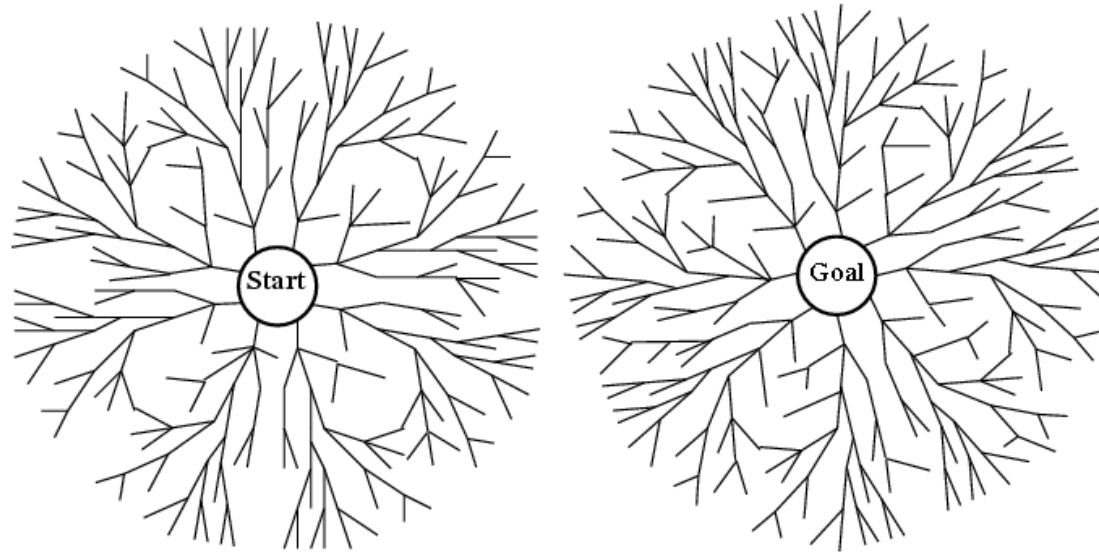
Not taking into account "how far" to the goal



Criteria	BFS	Uniform	DFS	DLS	IDS
Complete	Yes	Yes	No	No	Yes
Time	$O(b^d)$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$
Optimal	Yes	Yes	No	No	Yes

Bi-directional Search

Simultaneously search both forward from the initial state and backwards from the goal state - stop when searches meet.



- ❑ Problematic if there are lots of goal states
- ❑ How to meet in the middle?
- ❑ Need the efficient check on the status of the other search
- ❑ Could use two different search strategies

Part II

Informed Search

Generic Search Procedure

```
function SEARCH (problem, strategy)
  return solution or failure
```



Initialise the search tree with the initial state of the **problem**

Repeat

1. If (no candidate nodes can be expanded) then return failure
2. Choose a leaf node for expansion according to **strategy**
3. If (chosen node contains a goal state) then return solution
4. Else expand the node, by applying legal actions (operations) to the state of the node, and then adding newly generated node(s) to the search tree.

Redundancy

“Those who cannot remember the past are condemned to repeat it.”

--- George Santayana (1905)

“Search algorithms which cannot remember the past are condemned to repeat it”

--- Russell & Norvig (2009)

8-puzzle: possible oscillations between states

Grids: many paths to the same state

Improved Search Procedure

```
function SEARCH (problem, strategy)
  return solution or failure
```

Initialise the frontier with the initial state of the problem

Initialise the explored set to be empty

Repeat

1. If (the frontier is empty) then return failure
2. Choose a node from frontier for expansion according to strategy
3. If (chosen node contains a goal state) then return solution
4. Else

Add node to the explored set

Expand the node, by applying legal actions (operations) to the state of the node, and adding newly generated node(s) to the frontier, only if not in the frontier or explored set.



Recall from the First Session

- ❑ Uninformed search methods expand nodes based on “distance” from the root node.
 - Never look ahead to the goal.
 - e.g. in uniform-cost search we expand the cheapest path, but we never consider the cost of getting to the goal.
 - Advantage is that we have such information
- ❑ But, we often have some additional knowledge about the problem.
 - e.g. when traveling in Romania we know the straight distances between cities.

Informed Search

1	2	3
4	5	
7	8	6

Three possible moves of the blank space

One to the goal state



1	2	3
4		5
7	8	6



1	2	
4	5	3
7	8	6



1	2	3
4	5	6
7	8	



Informed Search via **Heuristics**

How do we choose the most promising move?

- ❑ Use heuristic functions to guide moves
- ❑ Specified as $h(n)$ or how promising is node n
- ❑ Lower values of $h(n)$ are better
- ❑ Estimating the cost from here to the goal
- ❑ Like uniform-cost search, but using the **evaluation function $f(n)$** rather than the path cost $g(n)$

Informed Search via **Heuristics** (cont'd)

How do we choose the most promising move?

- ❑ If $h(n1) < h(n2)$ then we guess that it is cheaper to reach the goal from $n1$ than from $n2$
- ❑ We require
 - $h(n) = 0$ when n is a goal node
 - $h(n) \geq 0$ for all other nodes

8-Puzzle Heuristics

h1: #tiles out of place

h2: Manhattan (city block) distance to goal

1	2	3
4	5	6
7		8

h1: 1
h2: 1

1	3	6
4	2	8
7		5

h1: 5
h2: 7

1	2	3
4	5	6
7	8	

h1: 0
h2: 0

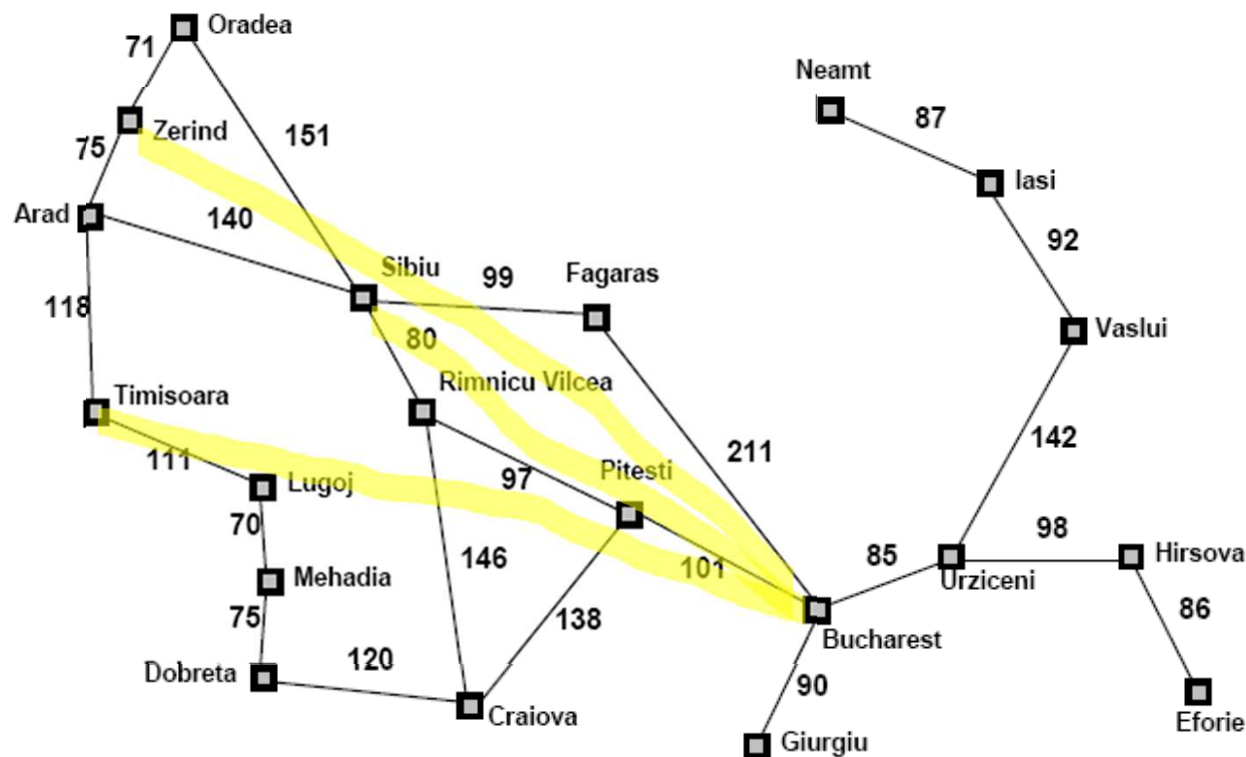
Greedy Best-First Search

- ❑ Use the heuristic function, $h(n)$, to rank the nodes in the frontier
- ❑ Search strategy
 - Expand node with the lowest h -value
- ❑ Heuristic version of DFS
 - Greedily trying to find the least-cost solution
- ❑ Not complete, not optimal
- ❑ Exponential in time and space
- ❑ Can be very efficient with a good heuristic

An Example of Greedy Best-First Search

Finding a route from **Arad** to **Bucharest**:

- ❑ Suppose straight-line distances are known
- ❑ Use them as heuristic values



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

An Example of Greedy Best-First Search (cont'd)

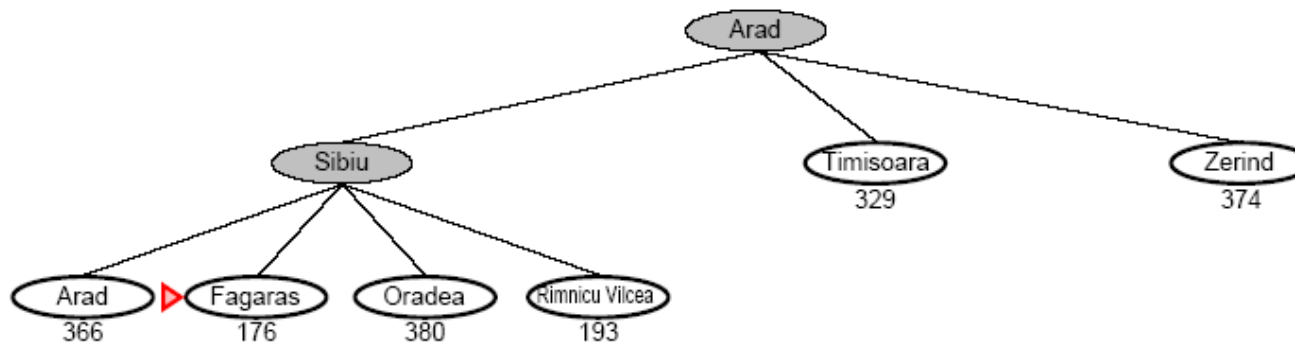
- Depth 0 (heuristic value shown under node)



- Depth 1:

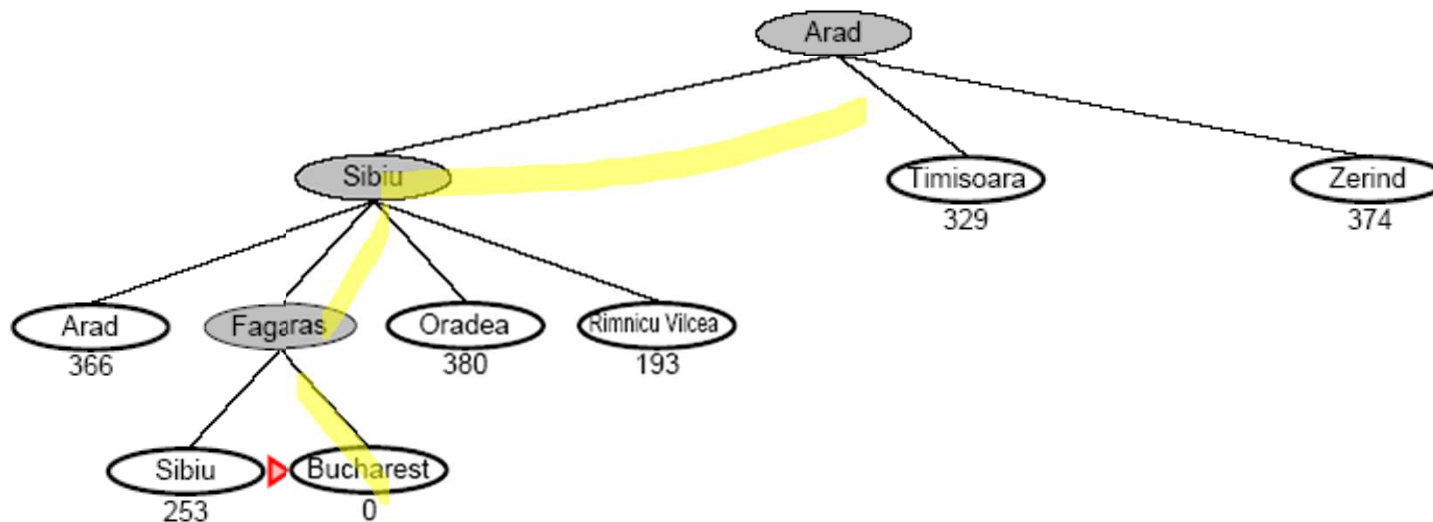


- Depth 2: Sibiu expanded due to lowest heuristic value



An Example of Greedy Best-First Search (cont'd)

□ Depth 3

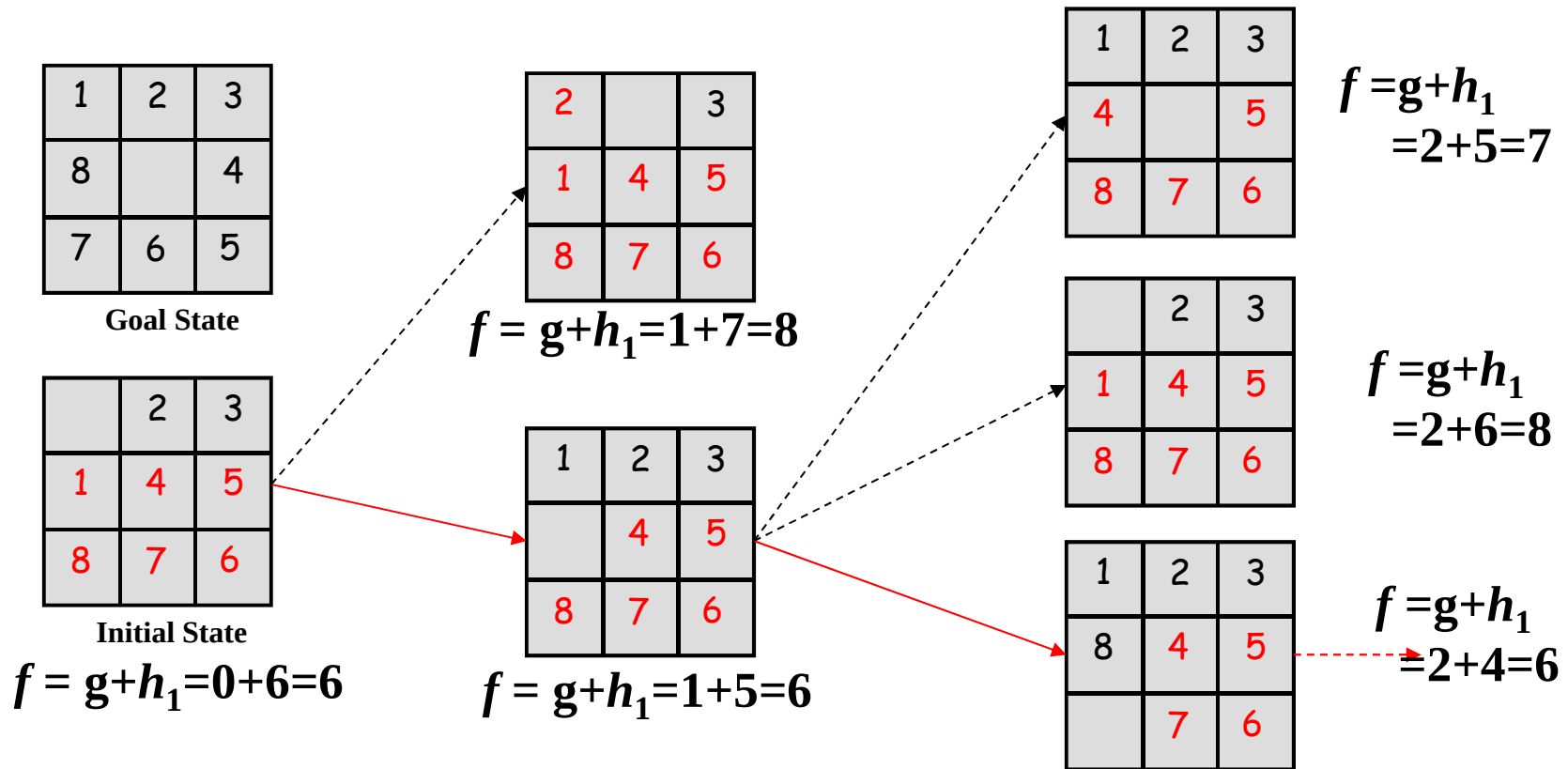


- Greedy best-first search is complete only if repeated states can be removed - otherwise it can get stuck in loops

A* Search

- $f(n) = h(n) + g(n)$
- Estimated cost of cheapest solution through n
- Like uniform-cost search but with $h(n) + g(n)$ instead of $g(n)$
- Optimal and complete when h is admissible
- h admissible if it never overestimates the cost
- Both $h1$ and $h2$ for the 8 puzzle are admissible
- $h2$ is more informed than $h1$ as $h2(n) \geq h1(n)$
- More informed heuristics expand less states

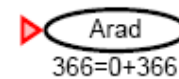
A* Search on 8-Puzzle Problems



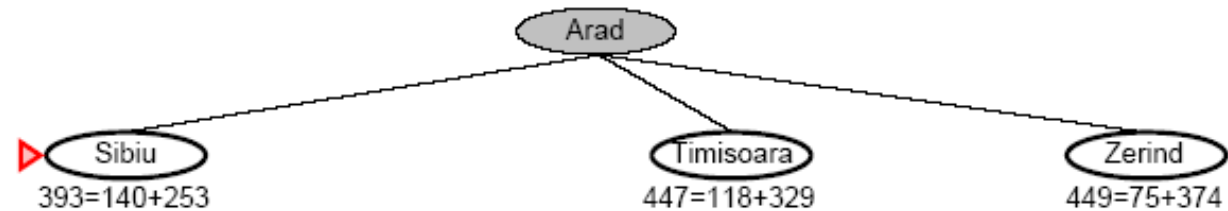
This example uses $g + h_1$ as evaluation function.

A* Search on Routine-Finding Problems

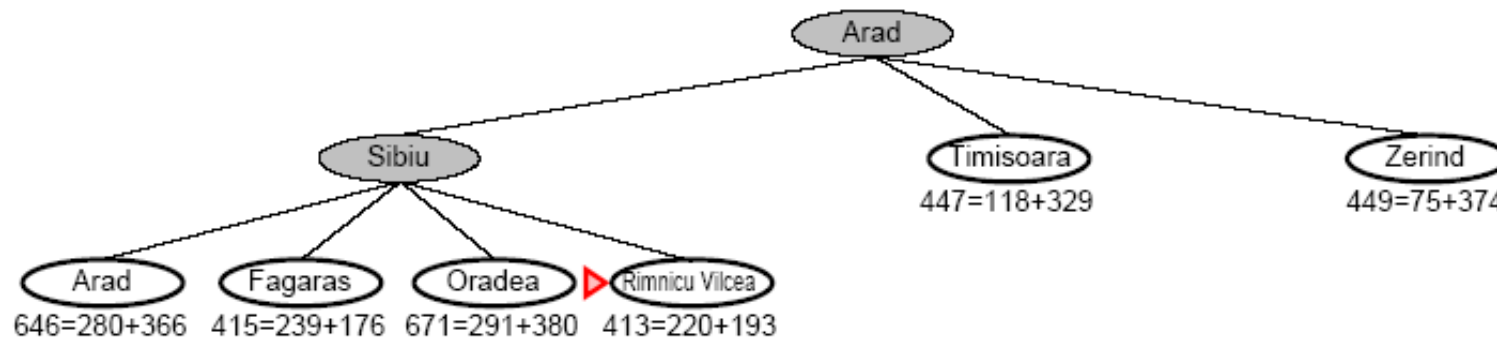
□ Depth 0:



□ Depth 1:

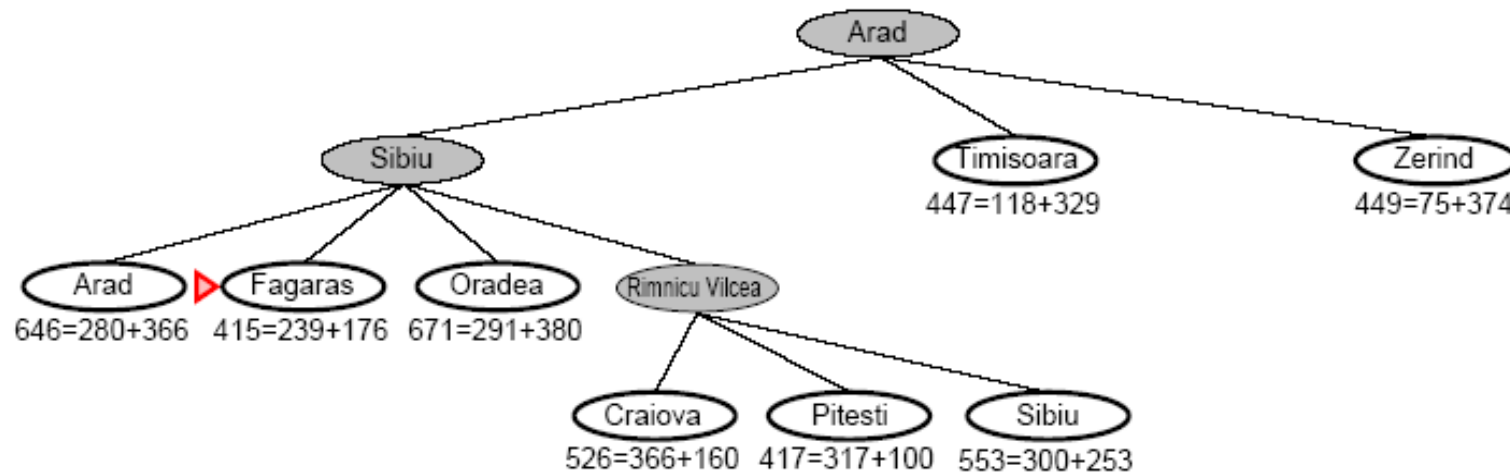


□ Depth 2:



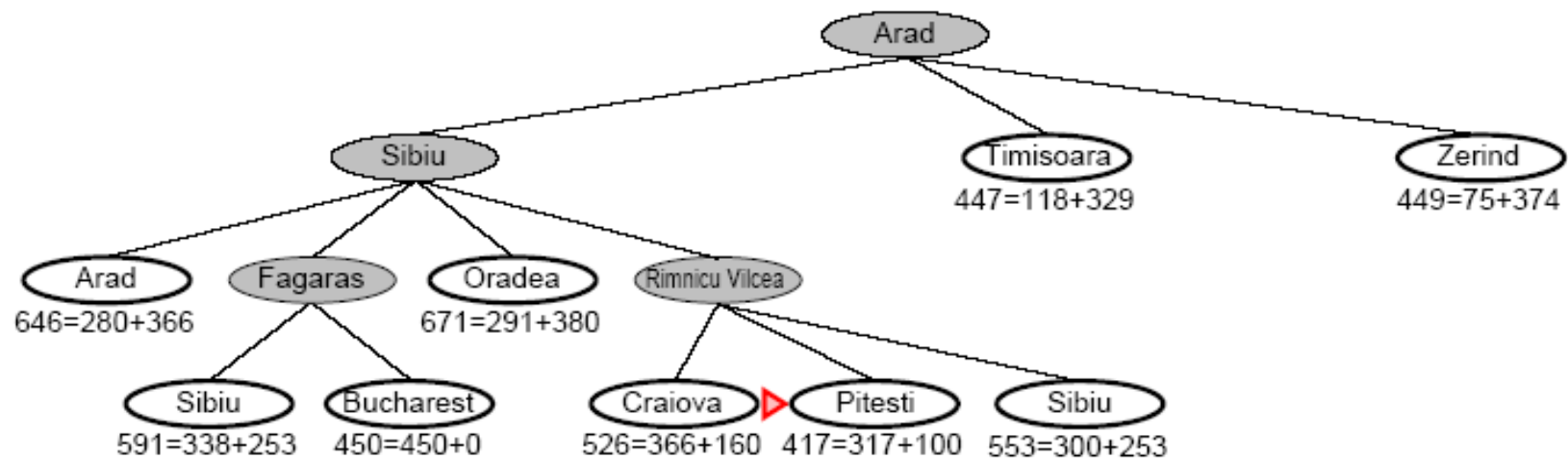
A* Search on Routine-Finding Problems (cont'd)

- Depth 3 - expanding Rimnicu Vilcea, not Fagaras



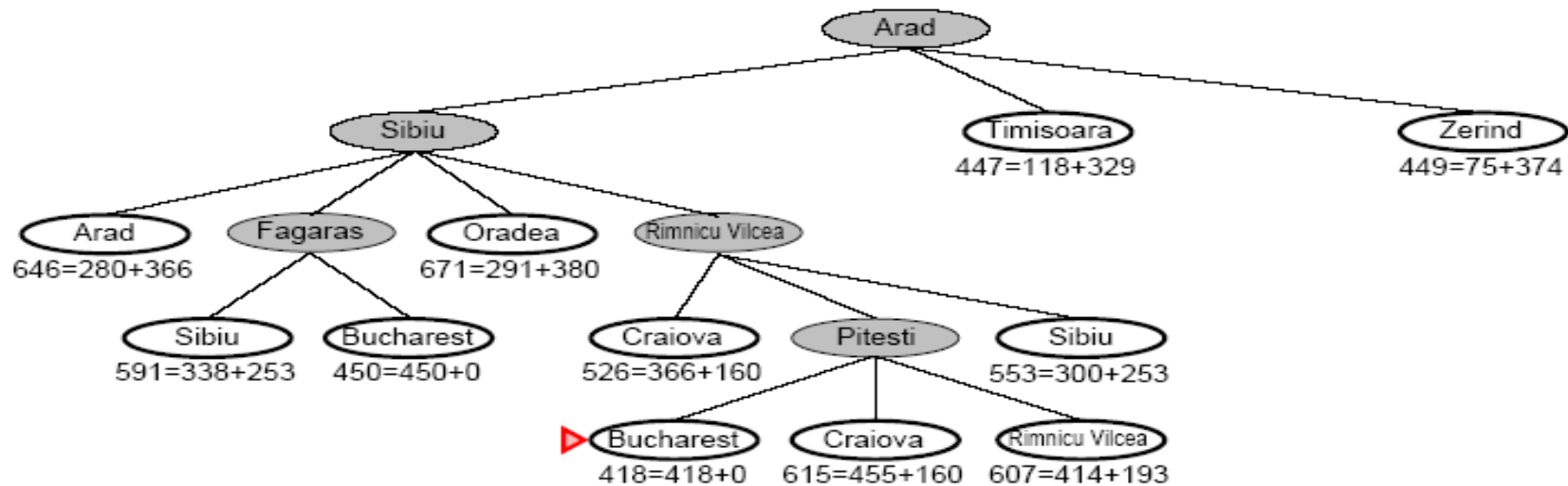
A* Search on Routine-Finding Problems (cont'd)

- Depth 3 (cont'd) - Fagaras is expanded because now it has the least evaluation value among the unexpanded nodes.



A* Search on Routine-Finding Problems (cont'd)

- Depth 4 - Pitesti will be expanded next because its value is even lower than Bucharest.



A* Search on Routine-Finding Problems (cont'd)

- ❑ The second Bucharest has the least evaluation value and thus it is the destination city. Therefore, the route is Arad - Sibiu - Rimnicu Vilcea - Pitesti - Bucharest.
- ❑ It is the optimal solution.
- ❑ It is cheaper than the route:
Arad - Sibiu - Fagaras - Bucharest

Optimality of A* Search

For searching graphs, we require some condition stronger than admissibility:

- ❑ Consistency (monotonicity):
 - $h(n) \leq \text{cost}(n, n') + h(n')$, for all nodes n, n'
- ❑ Almost any admissible heuristic function will also be consistent.

A* graph-search with a consistent heuristic is **optimal**.

Properties of A* Search

- ❑ **Complete** if the heuristic is consistent
 - Along any path, **f** always increases (if a solution exists somewhere, the **f** value will eventually get to its cost)
- ❑ Exponential time complexity in the worst case
 - A good heuristic will help a lot here
 - $O(b^m)$ if the heuristic is perfect
- ❑ **Exponential space complexity**

Heuristic Functions

- ❑ A good heuristic function can make all the difference!
- ❑ How do we get heuristics?
 - One approach is to think of an **easier problem** ("relaxed problem") and let **$h(n)$** be the cost of reaching the goal in the easier problem.
 - Use solution cost in relaxed problem as heuristic for the original problem.

Heuristic Functions of 8-Puzzle Problems

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Relax the game:

Which one is the best?

Manhattan distance

- ❑ Can move tile from position A to position B if A is next to B (ignore whether or not position is blank)
- ❑ Can move tile from position A to position B if B is blank (ignore adjacency)
- ❑ Can move tile from position A to position B

Misplaced tile heuristic

Cannot Tackle These Yet

Chance:



Infinite states:



Games against
adversary:



Hidden states:



All of the
above



Conclusion

We have studies:

- Uninformed search

- BFS
- DFS
- DLS
- IDS
- BDS

- Informed search

- Greedy BFS
- A*

